Solution: Removing Minimum and Maximum From Array

Problem Statement

Determine the `minimum` number of `deletions` required to remove the smallest and the largest elements from an array of integers.

In each `deletion`, you are allowed to remove either the `first (leftmost)` or the `last (rightmost)` element of the array.

Examples

1. **Example 1**:
   - Input: `[3, 2, 5, 1, 4]`
   - Expected Output: `3`
   - Justification: The smallest element is `1` and the largest is `5`. Removing `4`, `1`, and then `5` (or `5`, `4`, and then `1`) in three moves is the most efficient strategy.
2. **Example 2**:
   - Input: `[7, 5, 6, 8, 1]`
   - Expected Output: `2`
   - Justification: Here, `1` is the smallest, and `8` is the largest. Removing `1` and then `8` in two moves is the optimal strategy.
3. **Example 3**:
   - Input: `[2, 4, 10, 1, 3, 5]`
   - Expected Output: `4`
   - Justification: The smallest is `1` and the largest is `10`. One strategy is to remove `2`, `4`, `10`, and then `1` in four moves.

Solution

To solve this problem, identify the positions of the minimum and maximum elements in the array. Then, calculate the distance of these elements from both ends of the array. The key is to find the shortest path to remove both elements, considering three scenarios: removing both from the start, both from the end, one from the start and the other from the end. The minimum number of steps among these scenarios is the answer.

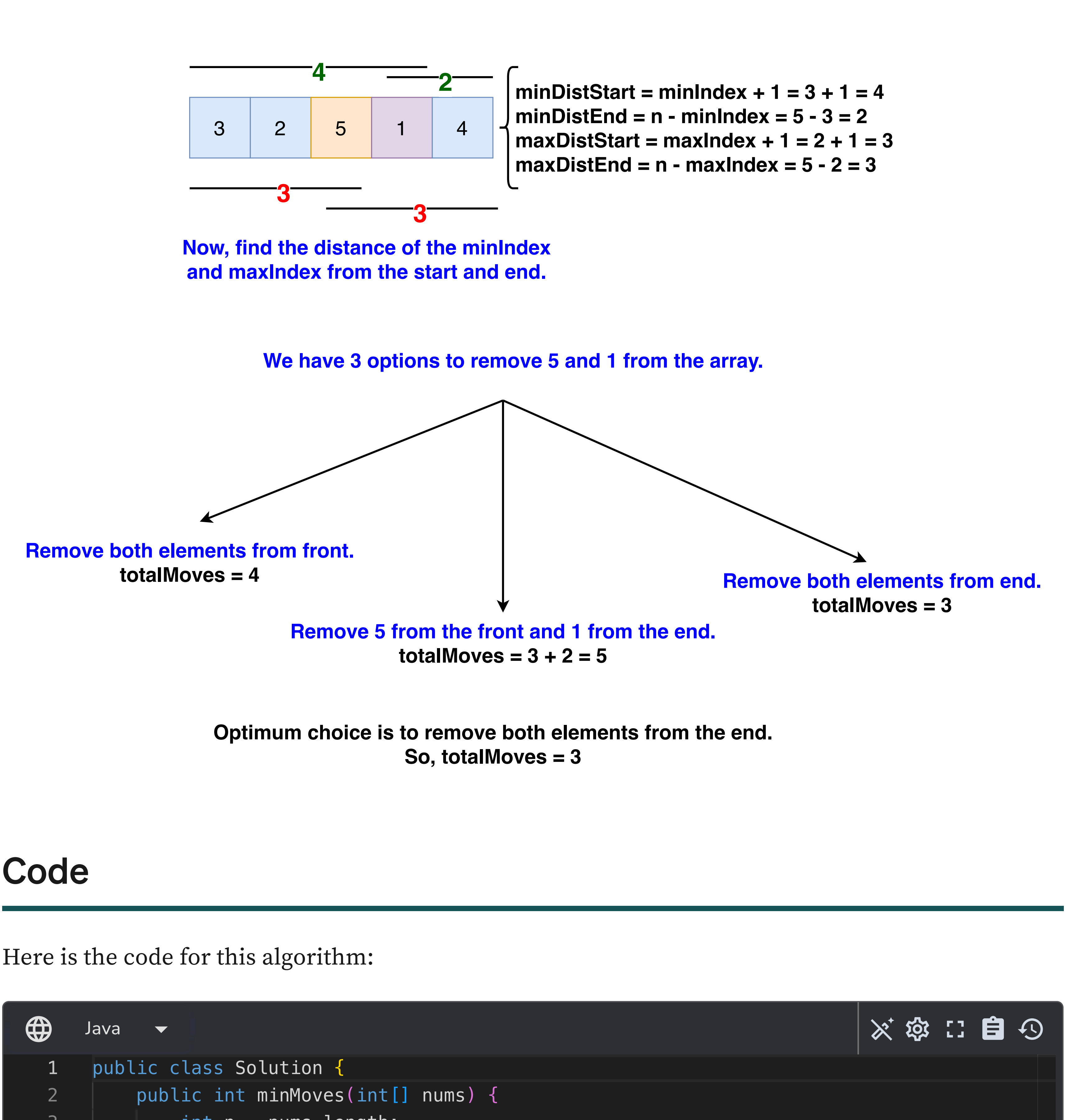
The steps are:

1. **Find Indices of Minimum and Maximum Elements**:
   - Iterate through the array to find the indices of the minimum and maximum elements. This step is essential because these positions dictate the strategy for removal.
2. **Calculate Distances**:
   - Calculate the distances of the minimum and maximum elements from both ends of the array.
3. **Evaluate Removal Strategies**:
   - There are several strategies to consider, and the goal is to find the one that requires the fewest moves:
     - Removing the minimum and maximum elements starting from the same end of the array.
     - Removing one of them from the start and the other from the end.
   - To find the optimal strategy, compare the total number of moves for each approach. This comparison involves summing up the relevant distances and determining the minimum sum.
4. **Return the Minimum Number of Moves**:
   - The final step is to return the smallest number of moves required among all evaluated strategies.

Algorithm Walkthrough

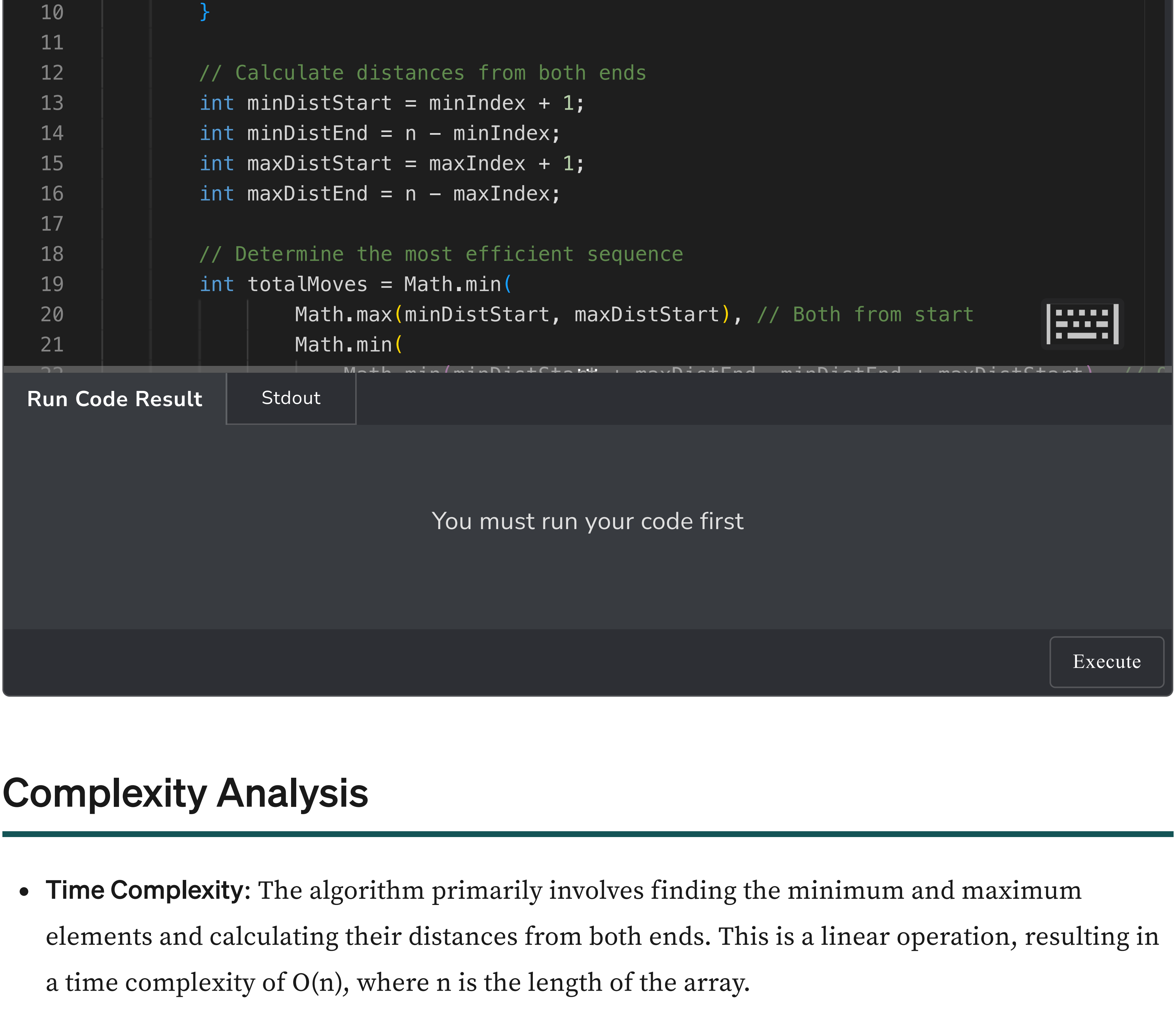
For the input `[3, 2, 5, 1, 4]`:

1. **Identify the Smallest and Largest Elements:**
   - Smallest element: `1` at index `3`.
   - Largest element: `5` at index `2`.
2. **Calculate Distances from Both Ends:**
   - Distance of `1` from the start: `4`.
   - Distance of `1` from the end: `2`.
   - Distance of `5` from the start: `3`.
   - Distance of `5` from the end: `3`.
3. **Determine the Most Efficient Removal Sequence:**
   - Option 1: Remove elements from start to reach `5` and then from the end to reach `1`. Total moves = `3 (for 5) + 2 (for 1) = 5`.
   - Option 2: Remove both elements from the end. Total moves to reach at `5` from the end is equal to `3`.
   - Option 3: Remove both elements from the start. Total moves to reach at `1` from the end is equal to `4`.
4. **Choose the Optimal Sequence:**
   - Option 2 provides the optimal result. The total number of moves required is `3`.


Input: nums = [3, 2, 5, 1, 4]
Largest element
Smallest element
3 2 5 1 4
maxIndex = 2; minIndex = 3
4
2
3 2 5 1 4
3
3
minDistStart = minIndex + 1 = 3 + 1 = 4
minDistEnd = n - minIndex = 5 - 3 = 2
maxDistStart = maxIndex + 1 = 2 + 1 = 3
maxDistEnd = n - maxIndex = 5 - 2 = 3
Now, find the distance of the minIndex and maxIndex from the start and end.
We have 3 options to remove 5 and 1 from the array.
Remove both elements from front.
totalMoves = 4
Remove 5 from the front and 1 from the end.
totalMoves = 3 + 2 = 5
Remove both elements from end.
totalMoves = 3
Optimum choice is to remove both elements from the end.
So, totalMoves = 3

Code

Here is the code for this algorithm:



```java
public class Solution {
    public int minMoves(int[] nums) {
        int n = nums.length;
        int minIndex = 0, maxIndex = 0;

        // Find the indexes of the minimum and maximum elements
        for (int i = 0; i < n; i++) {
            if (nums[i] < nums[minIndex]) minIndex = i;
            if (nums[i] > nums[maxIndex]) maxIndex = i;
        }

        // Calculate distances from both ends
        int minDistStart = minIndex + 1;
        int minDistEnd = n - minIndex;
        int maxDistStart = maxIndex + 1;
        int maxDistEnd = n - maxIndex;

        // Determine the most efficient sequence
        int totalMoves = Math.min(
                Math.max(minDistStart, maxDistStart), // Both from start
                Math.min(
```

Run Code Result
Stdout

You must run your code first

Complexity Analysis

- **Time Complexity:** The algorithm primarily involves finding the minimum and maximum elements and calculating their distances from both ends. This is a linear operation, resulting in a time complexity of O(n), where n is the length of the array.
- **Space Complexity:** Since no additional data structures are used that grow with the input size, the space complexity is O(1).

← Previous
Removing Minimum And Maximum From Array (Medium)

Next →
Introduction To Divide And Conquer Algorithm

☐ Mark as Completed